

Ngẫu nhiên hoá (Randomized Algorithms)

PHỤ THUỘC A/05 cho kỳ vọng tuyến tính và các bất đẳng thức đuôi; B/08 và B/09 cho hai ứng dụng đã gặp; C/14 cho quicksort ngẫu nhiên.

TÓM TẮT

Ngẫu nhiên hoá đã xuất hiện bốn lần trong quyển sách này — băm phổ dụng, treap, chốt quicksort, ước lượng cây quay lui — và chương này phát biểu nguyên lý chung đằng sau cả bốn: *khi trường hợp xấu phụ thuộc vào dữ liệu, hãy chuyển sự phụ thuộc sang một nguồn ngẫu nhiên mà bạn kiểm soát*. Đây không phải sự cầu may: kết quả là các đảm bảo đúng cho mọi dữ liệu vào, kể cả dữ liệu do đối thủ chọn. Chương phân biệt hai loại thuật toán ngẫu nhiên, đi qua các đại diện — kiểm tra nguyên tố, kiểm tra tích ma trận, lát cắt nhỏ nhất của Karger, lấy mẫu dòng dữ liệu — và nói về cách khuếch đại xác suất đúng bằng cách lặp lại. Nếu chỉ nhớ một điều: ngẫu nhiên hoá đôi “luôn đúng, đôi khi rất chậm” lấy “gần như luôn nhanh, và bạn kiểm soát được xác suất sai”.

KIẾN THỨC CẦN CÓ TRƯỚC

- **thuật toán Las Vegas** — luôn cho kết quả đúng; thời gian chạy là biến ngẫu nhiên. Quicksort ngẫu nhiên và treap thuộc loại này.
- **thuật toán Monte Carlo** — luôn chạy trong thời gian định trước; kết quả có thể sai với xác suất nhỏ. Kiểm tra nguyên tố Miller–Rabin thuộc loại này.
- **khuếch đại xác suất** — lặp lại một thuật toán Monte Carlo k lần độc lập; nếu mỗi lần sai với xác suất p thì k lần cùng sai chỉ với xác suất p^k — giảm theo hàm mũ.
- **lấy mẫu hồ chứa** — kỹ thuật chọn k phần tử đều ngẫu nhiên từ một dòng dữ liệu chưa biết độ dài, chỉ dùng $O(k)$ bộ nhớ.
- **khử ngẫu nhiên** — quá trình biến một thuật toán ngẫu nhiên thành tất định, thường bằng cách thay nguồn ngẫu nhiên thật bằng một họ hàm nhỏ.
- **đối thủ thích ứng** — mô hình trong đó dữ liệu được chọn sau khi biết thuật toán — lý do mọi phân tích “trung bình theo dữ liệu” sụp đổ trong thực tế.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao ngẫu nhiên hoá lại cho đảm bảo *mạnh hơn* phân tích trung bình, dù cả hai đều nói về xác suất?
- Q2. Một thuật toán trả lời sai với xác suất một phần tư nghe rất tệ. Vì sao nó vẫn dùng được trong thực tế?
- Q3. Kiểm tra xem $A \times B = C$ có đúng không mà không nhân ma trận — làm sao có thể nhanh hơn phép nhân?
- Q4. Trong lập trình thi đấu, vì sao dùng hàm sinh số ngẫu nhiên cố định hạt giống lại nguy hiểm?
- Q5. Thuật toán lát cắt nhỏ nhất của Karger có ngẫu nhiên các cạnh và thường cho kết quả sai. Vì sao nó vẫn là một thuật toán tốt?

Ngẫu nhiên hoá là chủ đề dễ bị hiểu sai nhất trong quyển sách này, vì cái tên gợi ý sự may rủi. Thực tế thì ngược lại: nó là công cụ để *loại bỏ* may rủi.

Hãy nhớ lại tình huống lặp đi lặp lại ở phần B. Bảng băm nhanh “trung bình”, nhưng trung bình đó giả định dữ liệu ngẫu nhiên — và dữ liệu thật thì không ngẫu nhiên, thậm chí có thể do người khác chọn để hại bạn. Cây tìm kiếm nhị phân cân bằng “trung bình”, nhưng dữ liệu đã sắp sẵn là chuyện thường ngày. Quicksort nhanh “trung bình”, với cùng lỗ hổng. Trong cả ba trường hợp, phân tích trung bình là một lời hứa dựa trên giả định về dữ liệu, và giả định đó không do bạn kiểm soát.

Ngẫu nhiên hoá sửa đúng chỗ đó: thay vì hy vọng dữ liệu ngẫu nhiên, hãy *tự tạo ra ngẫu nhiên* ở chỗ bạn kiểm soát — chọn hàm băm, chọn độ ưu tiên, chọn chốt. Khi đó hành vi của thuật toán không còn phụ thuộc thứ tự dữ liệu, và đảm bảo trở thành đúng cho mọi dữ liệu vào. Đây là khác biệt then chốt so với phân tích trung bình, và nó là câu trả lời cho câu hỏi mở đầu thứ nhất.

1.1 Hai loại và cách chọn giữa chúng

Phân biệt dưới đây theo khuôn Las Vegas / Monte Carlo của Motwani và Raghavan[1].

Las Vegas luôn đúng, thời gian chạy ngẫu nhiên. Bạn không bao giờ phải lo về tính đúng; bạn chỉ phải chấp nhận rằng thỉnh thoảng nó chạy lâu hơn. Quicksort với chốt ngẫu nhiên, treap, và thuật toán chọn phần tử thứ k đều thuộc loại này.

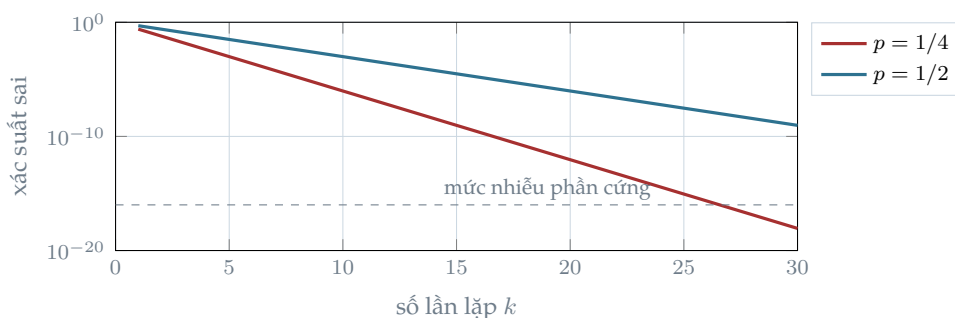
Monte Carlo luôn nhanh, kết quả có thể sai. Bạn kiểm soát được xác suất sai và làm nó nhỏ tùy ý bằng cách lặp lại. Kiểm tra nguyên tố, kiểm tra tích ma trận, băm chuỗi đều thuộc loại này.

Có một quy tắc chuyển đổi hữu ích: nếu kết quả *kiểm chứng được nhanh*, thì mọi thuật toán Monte Carlo biến thành Las Vegas bằng cách lặp lại tới khi kiểm chứng thành công. Ví dụ: một thuật toán ngẫu nhiên đề xuất một cách ghép cặp; kiểm tra cách ghép có hợp lệ hay không thì rẻ, nên cứ thử lại tới khi được. Ngược lại, khi kết quả không kiểm chứng được — “số này có phải nguyên tố không” — thì bạn buộc phải sống với xác suất sai.

1.2 Khuếch đại xác suất: vì sao “sai một phần tư” vẫn dùng được

Đây là kỹ thuật làm cho toàn bộ hướng tiếp cận trở nên khả thi, và nó dựa trên một tính toán rất đơn giản.

Giả sử thuật toán trả lời sai với xác suất tối đa $1/4$, và các lần chạy độc lập. Chạy k lần và lấy kết quả xuất hiện nhiều nhất; xác suất tất cả k lần cùng sai là $(1/4)^k$. Với $k = 30$, con số đó nhỏ hơn 10^{-18} — tức là nhỏ hơn xác suất phần cứng của bạn bị lỗi bit trong lúc chạy.



Hình 1.1. Khuếch đại xác suất. Xác suất sai giảm theo *hàm mũ* của số lần lặp, nên vài chục lần là đủ để đưa nó xuống dưới mức mà các nguồn sai khác — kể cả lỗi phần cứng — đã chi phối. Điều kiện bắt buộc: các lần chạy phải độc lập.

Điều đáng nhấn mạnh về mặt kỹ thuật: đây là dạng đơn giản nhất của các bất đẳng thức đuôi ở chương A/05, và nó cho thấy vì sao Chernoff quan trọng — xác suất sai giảm theo *hàm mũ* của số lần lặp, chứ không theo tỉ lệ nghịch. Chính điều đó cho phép ta trả một chi phí nhỏ (vài chục lần lặp) để đổi lấy độ tin cậy thực tế tuyệt đối.

Một điều kiện dễ bị quên: các lần chạy phải *độc lập*. Nếu bạn lặp lại thuật toán với cùng hạt giống, hoặc với các lựa chọn tương quan nhau, thì phép nhân xác suất không còn đúng và bạn chỉ đang lãng phí thời gian.

CẦN NHỚ

Trong lập trình thi đấu, luôn khởi tạo bộ sinh ngẫu nhiên bằng một nguồn không đoán trước — thời gian đồng hồ hệ thống ở độ phân giải cao, hoặc một giá trị phụ thuộc địa chỉ bộ nhớ. Dùng hạt giống cố định biến thuật toán ngẫu nhiên của bạn thành tất định, và khi đó người ra đề (hoặc người viết test phá) có thể dựng dữ liệu chạm đúng trường hợp xấu — đúng lỗ hổng mà ngẫu nhiên hoá lẽ ra đã bịt.

1.3 Bốn đại diện đáng biết

Kiểm tra nguyên tố. Muốn biết n có phải số nguyên tố, thử chia tới $\sqrt{n}$ là $O(\sqrt{n})$ — quá chậm với số 60 bit. Kiểm tra Miller–Rabin chọn ngẫu nhiên vài “nhân chứng” và dùng một tính chất số học để phát hiện hợp số; mỗi nhân chứng phát hiện hợp số với xác suất ít nhất $3/4$, nên vài chục nhân chứng cho độ tin cậy tuyệt đối trong thực tế. Đây là thuật toán chạy trong mọi thư viện mật mã, và là ví dụ chuẩn cho việc ngẫu nhiên hoá không phải giải pháp tạm mà là công cụ sản xuất.

Kiểm tra tích ma trận (Freivalds). Cho ba ma trận A, B, C cỡ $n \times n$, hỏi $AB = C$ có đúng không. Tính AB tốn $O(n^3)$. Mẹo: chọn một vector ngẫu nhiên r gồm các bit, và so $A(Br)$ với Cr — cả hai về chỉ cần các phép nhân ma trận–vector, tốn $O(n^2)$. Nếu $AB \neq C$ thì phép so sai với xác suất tối đa $1/2$, và lặp vài chục lần là đủ. Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và nó minh hoạ một ý sâu hơn: *kiểm tra một đẳng thức rẻ hơn tính ra nó*, vì ta chỉ cần kiểm nó trên một hướng ngẫu nhiên thay vì trên mọi hướng.

Lát cắt nhỏ nhất của Karger. Lặp lại việc co một cạnh ngẫu nhiên cho tới khi còn hai đỉnh; các cạnh còn lại là một lát cắt. Xác suất ra đúng lát cắt nhỏ nhất chỉ khoảng $2/n^2$ — rất thấp — nhưng vì đó là xác suất chứ không phải sự bế tắc, ta chỉ cần lặp $O(n^2 \log n)$ lần để có độ tin cậy cao. Điều làm thuật toán này nổi tiếng không phải hiệu năng mà là sự đơn giản: nó không dùng luồng, không dùng lý thuyết nào ngoài việc co cạnh, và nó chứng minh rằng một cách tiếp cận “ngây thơ có ngẫu nhiên” vẫn có thể cho kết quả sâu sắc.

Lấy mẫu hồ chứa. Chọn đều ngẫu nhiên k phần tử từ một dòng dữ liệu mà bạn không biết trước độ dài và không lưu được. Giữ k phần tử đầu; với phần tử thứ $i > k$, giữ nó với xác suất k/i và nếu giữ thì thay thế một phần tử ngẫu nhiên trong hồ. Chứng minh tính đúng bằng quy nạp là một bài tập đẹp, và kỹ thuật này là công cụ chuẩn trong xử lý dòng dữ liệu (D/24).

1.4 Ngẫu nhiên hoá như một mẫu thiết kế

Nhìn lại toàn bộ quyển sách, ngẫu nhiên hoá xuất hiện dưới bốn vai trò khác nhau. Nhận ra bốn vai trò này giúp bạn biết khi nào nên nghĩ tới nó.

Vai trò 1 — phá vỡ trường hợp xấu do dữ liệu. Băm với hạt giống ngẫu nhiên (B/08), treap (B/09), chốt quicksort (C/14). Dấu hiệu: thuật toán của bạn nhanh trừ khi dữ liệu có cấu trúc đặc biệt.

Vai trò 2 — kiểm tra rẻ hơn tính toán. Freivalds, băm chuỗi (B/12), so sánh hai đa thức bằng cách thay một giá trị ngẫu nhiên. Dấu hiệu: bạn cần kiểm một đẳng thức chứ không cần biết giá trị.

Vai trò 3 — lấy mẫu thay cho vét cạn. Ước lượng kích thước cây quay lui (C/13), lấy mẫu hồ chứa, các phương pháp Monte Carlo cho tích phân. Dấu hiệu: không gian quá lớn để duyệt nhưng bạn chỉ cần một ước lượng.

Vai trò 4 — thoát khỏi cấu trúc xấu. Karger, khởi động lại nhiều lần trong tìm kiếm cục bộ (C/19), RANSAC trong thị giác máy tính. Dấu hiệu: có nhiều lời giải khả dĩ và bạn chỉ cần tìm được một cái tốt, chứ không cần chứng minh nó tốt nhất.

CẠM BẮY

Ngẫu nhiên hoá không sửa được một thuật toán *sai*. Nếu ý tưởng của bạn có phần ví dụ về mặt logic thì việc thêm ngẫu nhiên chỉ làm nó sai một cách khó dự đoán hơn. Ngẫu nhiên hoá chỉ giải quyết vấn đề *phân phối dữ liệu*, không giải quyết vấn đề tính đúng. Trước khi thêm ngẫu nhiên, hãy chắc chắn bạn đang ở đúng loại vấn đề.

1.5 Gốc rễ: một ván bài trong bệnh viện và bài toán số nguyên tố

Phương pháp Monte Carlo có một nguồn gốc được kể lại khá thống nhất. Năm 1946, Stanislaw Ulam đang nằm dưỡng bệnh và chơi bài một mình. Ông tự hỏi xác suất để một ván bài kiểu solitaire thắng là bao nhiêu, và nhận ra rằng việc tính chính xác bằng tổ hợp là gần như bất khả thi — trong khi việc *chơi thử một trăm ván và đếm* thì rất dễ. Ý tưởng đó, mang sang cho các bài toán khuếch tán neutron mà ông và von Neumann đang làm tại Los Alamos, trở thành phương pháp Monte Carlo. Cái tên được đặt theo sông bạc ở Monaco, nơi một người họ hàng của Ulam hay lui tới.

Bài học phương pháp ở đây rất đáng nhớ và nó áp dụng cho cả việc giải thuật toán: khi một đại lượng khó tính chính xác nhưng dễ *lấy mẫu*, hãy đổi bài toán tính sang bài toán ước lượng. Đây chính là vai trò thứ ba ở mục 5.

Nhánh thuật toán ngẫu nhiên trong khoa học máy tính bắt đầu nghiêm túc vào giữa thập niên 1970 với bài toán kiểm tra nguyên tố. Solovay và Strassen, rồi Rabin, đưa ra các thuật toán xác suất nhanh cho một bài toán mà khi đó chưa ai biết có lời giải đa thức tất định hay không. Điều này đặt ra một câu hỏi lý thuyết quan trọng: ngẫu nhiên có thực sự cho thêm sức mạnh tính toán, hay mọi thuật toán ngẫu nhiên đều khử ngẫu nhiên được? Câu chuyện có một kết cục thú vị: năm 2002, ba nhà nghiên cứu Ấn Độ công bố một thuật toán *tất định* thời gian đa thức cho kiểm tra nguyên tố, giải quyết đúng bài toán mở đầu của cả nhánh — dù trong thực tế thì các thuật toán ngẫu nhiên vẫn nhanh hơn nhiều và vẫn là thứ được dùng.

Câu hỏi tổng quát vẫn còn mở nhưng nghiêng về một phía: phần lớn giới nghiên cứu tin rằng ngẫu nhiên *không* cho thêm sức mạnh về mặt lớp độ phức tạp, tức là mọi thuật toán ngẫu nhiên đa thức đều có phiên bản tất định đa thức. Điều đó không làm ngẫu nhiên hoá mất giá trị — các phiên bản tất định thường phức tạp và chậm hơn nhiều bậc trong thực tế — nhưng nó nói rằng sức mạnh thật sự của ngẫu nhiên nằm ở *sự đơn giản*, không ở khả năng tính toán.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
1946 — Ulam, von Neumann	Tính xác suất thắng của một ván bài; rồi bài toán khuếch tán neutron ở Los Alamos.	Phương pháp Monte Carlo: khi khó tính chính xác nhưng dễ lấy mẫu, hãy đổi sang ước lượng.
1975–76 — Solovay, Strassen, Rabin	Kiểm tra số lớn có nguyên tố không — chưa ai biết lời giải đa thức tất định.	Kiểm tra nguyên tố xác suất; khai sinh nhánh thuật toán ngẫu nhiên trong khoa học máy tính.
1977 — Freivalds	Kiểm tra $AB = C$ mà không muốn tốn $O(n^3)$.	Kiểm bằng một vector ngẫu nhiên trong $O(n^2)$; ý tưởng “kiểm rẻ hơn tính”.
1979 — Carter & Wegman	Bảng băm nhanh trung bình nhưng không có đảm bảo với dữ liệu thù địch.	Băm phổ dụng: ngẫu nhiên nằm ở lựa chọn hàm, không ở dữ liệu (B/08).
1993 — Karger	Tìm lát cắt nhỏ nhất mà không dùng luồng.	Co cạnh ngẫu nhiên; xác suất thành công thấp nhưng lặp lại được — minh hoạ rằng đơn giản cộng ngẫu nhiên vẫn cho kết quả sâu.
2002 — Agrawal, Kayal, Saxena	Kiểm tra nguyên tố có lời giải đa thức tất định không?	Có — nhưng chậm hơn bản ngẫu nhiên nhiều bậc trong thực tế. Sức mạnh của ngẫu nhiên nằm ở sự đơn giản, không ở lớp độ phức tạp.

1.6 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Thị giác máy tính, SLAM	RANSAC: lấy mẫu ngẫu nhiên tập con nhỏ để ước lượng mô hình	Vai trò 4 ở mục 5, và là ứng dụng ngẫu nhiên hoá quan trọng nhất trong ngành này. Cùng lập luận khuếch đại: xác suất một lần lấy mẫu “sạch” thì thấp, nhưng lặp đủ nhiều thì gần như chắc chắn có ít nhất một lần sạch.
Ước lượng trạng thái	Bộ lọc hạt: biểu diễn phân phối bằng một tập mẫu	Monte Carlo áp cho bài toán ước lượng liên tục. Cùng tinh thần với mục 4: khi phân phối khó mô tả bằng công thức, hãy mô tả nó bằng mẫu. Cạm bẫy đặc thù là suy thoái mẫu, cần lấy mẫu lại.
Thống kê	Bootstrap, MCMC, kiểm định hoán vị	Cùng ý tưởng gốc của Ulam: thay tính toán giải tích bằng mô phỏng lặp lại. Khác: ở đó mục tiêu là ước lượng độ bất định, ở đây là ước lượng một đại lượng.
Mật mã	Sinh khoá, kiểm tra nguyên tố, giao thức xác suất	Kiểm tra nguyên tố ngẫu nhiên chạy mỗi lần bạn tạo một khoá. Điểm khác biệt quan trọng: ở đó chất lượng nguồn ngẫu nhiên là vấn đề an ninh, không chỉ là vấn đề hiệu năng.
Học máy	Khởi tạo ngẫu nhiên, bỏ ngẫu nhiên nơ-ron, trộn dữ liệu, rừng ngẫu nhiên	Ngẫu nhiên ở đây phục vụ tính tổng quát hoá chứ không phục vụ tốc độ: nó phá vỡ sự tương quan và ngăn mô hình bám vào cấu trúc giả. Cùng nguyên lý phá vỡ cấu trúc xấu, khác mục tiêu.
Xử lý dòng dữ liệu	Lấy mẫu hồ chứa, sơ đồ đếm xấp xỉ, HyperLogLog	Ngẫu nhiên là điều kiện để làm việc với bộ nhớ nhỏ hơn dữ liệu nhiều bậc (D/24). Cùng đánh đổi: chấp nhận sai số nhỏ để đổi lấy khả năng chạy được.
Trò chơi và AI	Tìm kiếm cây Monte Carlo	Thay vì duyệt cây trò chơi đầy đủ, lấy mẫu các ván chơi tới cuối và thống kê. Cùng vai trò 3; đây là kỹ thuật đứng sau bước nhảy vọt của các chương trình chơi cờ vây.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì hai loại phân tích nói về hai nguồn ngẫu nhiên khác nhau. Phân tích trung bình giả định dữ liệu được rút từ một phân phối nào đó — một giả định về thể giới, và nó sai khi dữ liệu có cấu trúc hoặc do đối thủ chọn. Ngẫu nhiên hoá đặt nguồn ngẫu nhiên vào lựa chọn của thuật toán, thứ mà bạn kiểm soát và đối thủ không biết. Kết quả là một đảm bảo đúng cho mọi dữ liệu vào, không kèm giả định nào về phân phối dữ liệu.
- A2. Vì xác suất sai giảm theo hàm mũ khi lặp lại. Chạy k lần độc lập, xác suất tất cả cùng sai là $(1/4)^k$; với $k = 30$ thì nhỏ hơn 10^{-18} , tức là nhỏ hơn xác suất máy tính bị lỗi phần cứng trong lúc chạy. Điều kiện là các lần chạy phải thật sự độc lập — dùng lại cùng hạt giống hoặc các lựa chọn tương quan sẽ phá vỡ phép nhân xác suất và bạn chỉ đang lặp lại cùng một sai lầm.
- A3. Vì để kiểm một đẳng thức, ta không cần kiểm nó theo mọi hướng — chỉ cần theo một hướng ngẫu nhiên là đủ để phát hiện sai lệch với xác suất cao. Cụ thể, thay vì tính AB rồi so với C , ta chọn vector ngẫu nhiên r và so $A(Br)$ với Cr ; cả hai vế chỉ cần nhân ma trận với vector, tốn $O(n^2)$. Nếu hai ma trận khác nhau thì chúng khác nhau trên phần lớn các vector, nên xác suất bỏ sót là nhỏ và giảm theo hàm mũ khi lặp lại.
- A4. Vì hạt giống cố định làm thuật toán trở thành tất định, và mọi thứ tất định đều dựng được test phá. Người viết test chỉ cần chạy đúng chương trình của bạn để biết nó sẽ chọn chốt nào, chọn hàm băm nào, rồi dựng dữ liệu chạm đúng trường hợp xấu. Toàn bộ giá trị của ngẫu nhiên hoá nằm ở chỗ đối thủ không đoán được, nên hạt giống phải lấy từ nguồn không đoán trước như đồng hồ độ phân giải cao.
- A5. Vì thất bại của nó là thất bại có xác suất, không phải bế tắc, nên lặp lại giải quyết được. Xác suất một lần chạy ra đúng lát cắt nhỏ nhất là khoảng $2/n^2$; lặp $O(n^2 \log n)$ lần cho xác suất bỏ sót nhỏ tùy ý. Giá trị của nó không nằm ở hiệu năng — các thuật toán dựa trên luồng nhanh hơn — mà ở chỗ nó cho thấy một thủ tục cực kỳ đơn giản, không dùng lý thuyết nào ngoài việc co cạnh, vẫn giải được bài toán mà trước đó chỉ có lời giải phức tạp.

NÓI LẠI THẬT ĐƠN GIẢN

Có một hiểu lầm phổ biến: nghĩ rằng dùng số ngẫu nhiên là phó mặc cho may rủi. Thực ra thì ngược lại — người ta dùng ngẫu nhiên để *khỏi phụ thuộc vào may rủi*.

Hãy nhớ lại chuyện cái tủ hồ sơ ở chương về băm. Nếu quy tắc xếp hồ sơ của bạn là cố định thì người khác có thể mang tới toàn những hồ sơ rơi vào cùng một ngăn, và tủ của bạn thành vô dụng. Nhưng nếu mỗi sáng bạn trộn thêm một con số bí mật vào quy tắc thì họ không đoán được nữa. Bạn không hy vọng gặp may; bạn đã *loại bỏ* khả năng bị hại.

Cùng logic đó áp cho việc sắp xếp: chọn phần tử làm mốc một cách ngẫu nhiên thì không có dây đầu vào nào là “dây xấu” nữa, vì cái xấu bây giờ phụ thuộc vào đồng xu của bạn chứ không phụ thuộc vào dữ liệu.

Loại thứ hai của ngẫu nhiên hoá thì khác: chấp nhận trả lời sai đôi khi, để đổi lấy tốc độ. Nghe đáng lo, cho tới khi bạn làm phép tính: nếu mỗi lần trả lời sai với xác suất một phần tư, thì ba mươi lần liên tiếp cùng sai có xác suất nhỏ hơn xác suất máy tính của bạn hỏng giữa chừng. Ở mức đó, “có thể sai” trở thành một khái niệm không còn ý nghĩa thực tế.

Loại thứ ba là lấy mẫu, và nó đến từ một ý rất đời: nếu bạn muốn biết trong nồi canh đủ mặn chưa, bạn không uống cả nồi — bạn ném một thìa. Với điều kiện là đã khuấy đều. Toàn bộ nghệ thuật của các phương pháp lấy mẫu nằm ở đúng chữ “khuấy đều” đó.

Một câu: Đừng hy vọng dữ liệu ngẫu nhiên; hãy tự tạo ngẫu nhiên ở chỗ bạn kiểm soát.

Chỗ không rút gọn được: Chất lượng của nguồn ngẫu nhiên là chuyện thật, không phải chi tiết: một bộ sinh kém hoặc một hạt giống đoán được sẽ phá huỷ mọi đảm bảo ở trên.

Thuật ngữ / câu hỏi	Trả lời
●●● Nguyên lý cốt lõi	Khi trường hợp xấu phụ thuộc dữ liệu, chuyển sự phụ thuộc sang nguồn ngẫu nhiên bạn kiểm soát; đảm bảo trở thành đúng cho <i>mọi</i> dữ liệu vào.
●●● Ngẫu nhiên hoá khác phân tích trung bình	Trung bình giả định về phân phối <i>dữ liệu</i> (không kiểm soát được); ngẫu nhiên hoá đặt ngẫu nhiên vào <i>lựa chọn của thuật toán</i> .
●●● Las Vegas so với Monte Carlo	Las Vegas: luôn đúng, thời gian ngẫu nhiên. Monte Carlo: luôn nhanh, có thể sai. Kết quả kiểm chứng được nhanh thì chuyển Monte Carlo thành Las Vegas bằng cách lặp.
●●● Khuếch đại xác suất	Sai với xác suất p mỗi lần, lặp k lần độc lập thì p^k ; $k = 30$ với $p = 1/4$ cho dưới 10^{-18} . Bắt buộc: các lần chạy độc lập.
●●● Hạt giống trong thi đấu	Phải lấy từ nguồn không đoán trước (đồng hồ độ phân giải cao); hạt giống cố định biến thuật toán thành tất định và dựng test phá được.
●● Bốn vai trò của ngẫu nhiên hoá	Phá trường hợp xấu do dữ liệu; kiểm rẻ hơn tính; lấy mẫu thay vết cặn; thoát khỏi cấu trúc xấu.
●● Freivalds	Kiểm $AB = C$ bằng vector ngẫu nhiên trong $O(n^2)$ thay vì $O(n^3)$; ý tưởng “kiểm một đẳng thức chỉ cần một hướng ngẫu nhiên”.
●● Miller–Rabin	Chọn nhân chứng ngẫu nhiên; mỗi nhân chứng phát hiện hợp số với xác suất $\geq 3/4$; là thuật toán dừng trong mọi thư viện mật mã.
●● Karger	Co cạnh ngẫu nhiên tới khi còn hai đỉnh; xác suất đúng $\approx 2/n^2$, lặp $O(n^2 \log n)$ lần. Giá trị nằm ở sự đơn giản, không ở tốc độ.
●● Lấy mẫu hồ chứa	Giữ k phần tử đầu; phần tử thứ $i > k$ được giữ với xác suất k/i , thay thế ngẫu nhiên một phần tử trong hồ. Bộ nhớ $O(k)$, không cần biết độ dài dòng.
●● Ngẫu nhiên không sửa được cái gì	Không sửa được thuật toán <i>sai</i> về logic; nó chỉ giải quyết vấn đề phân phối dữ liệu.
● Ngẫu nhiên và lớp độ phức tạp	Phần lớn giới nghiên cứu tin rằng ngẫu nhiên không thêm sức mạnh về lớp; giá trị thật của nó là sự đơn giản và hằng số nhỏ.

●●● phải thuộc ●● nên nắm ● biết là có

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Tôi nên dùng bộ sinh ngẫu nhiên nào trong C++?

| Dùng `mt19937` (hoặc `mt19937_64` cho số 64 bit) với hạt giống lấy từ đồng hồ độ phân giải cao. Tránh `rand()`: chất lượng kém, khoảng giá trị nhỏ trên một số nền tảng, và cách lấy ngẫu nhiên trong một khoảng bằng phép chia dư gây lệch phân phối. Để lấy số ngẫu nhiên trong $[a, b]$, dùng `uniform_int_distribution` thay vì phép chia dư.

2. Trộn ngẫu nhiên mảng trước khi sắp có thực sự cần không?

| Cần, trong hai tình huống. Thứ nhất, khi bạn tự cài quicksort mà không chọn chốt ngẫu nhiên — lúc đó trộn trước là cách rẻ nhất để loại bỏ trường hợp xấu. Thứ hai, khi dùng hàm sắp xếp thư viện trong một kỳ thi mà bạn biết có test phá nhắm vào cài đặt cụ thể của thư viện đó. Chi phí là một dòng và $O(n)$, còn lợi ích là loại hẳn một lớp thất bại — một đánh đổi rất đáng.

3. Làm sao chứng minh một thuật toán ngẫu nhiên là đúng?

| Chia làm hai phần và đừng lẫn chúng. Phần tính đúng: với Las Vegas thì chứng minh như thuật toán tất định — bất biến, quy nạp ($A/04$); ngẫu nhiên chỉ ảnh hưởng thời gian. Với Monte Carlo thì phải chặn xác suất sai, thường bằng cách chỉ ra “một lựa chọn ngẫu nhiên là xấu chỉ khi rơi vào một tập nhỏ”. Phần chi phí: dùng kỳ vọng tuyến tính và các bất đẳng thức đuôi ở $A/05$, và nhớ nói rõ kỳ vọng lấy theo cái gì.

4. Nếu ngẫu nhiên hoá tốt như vậy, vì sao không dùng nó ở mọi nơi?

| Vì nó có ba cái giá. Một, kết quả không tái lập được giữa các lần chạy, gây khó cho gỡ lỗi — cách chữa là ghi lại hạt giống của mỗi lần chạy để tái hiện. Hai, với Monte Carlo, bạn phải chấp nhận xác suất sai khác không; trong các hệ thống quan trọng, điều này cần được nêu rõ chứ không được giấu. Ba, một số bài đơn giản là không có cấu trúc để ngẫu nhiên hoá khai thác. Ngẫu nhiên là công cụ, không phải liều thuốc chung.

5. Trong hệ thống thật, tôi phải cẩn thận gì với ngẫu nhiên?

| Ba điều. Một, ghi lại hạt giống để tái lập được sự cố — đây là điều khác biệt lớn nhất giữa hệ thống có kỷ luật và hệ thống không. Hai, phân biệt bộ sinh ngẫu nhiên thông thường với bộ sinh an toàn mật mã: dùng loại thứ nhất cho khoá hay mã phiên là một lỗ hổng bảo mật thật sự. Ba, trong hệ nhiều luồng, chia sẻ một bộ sinh giữa các luồng vừa gây tranh chấp vừa có thể phá tính độc lập — mỗi luồng nên có bộ sinh riêng với hạt giống khác nhau.

6. RANSAC có phải thuật toán ngẫu nhiên theo nghĩa của chương này không?

| Đúng, và nó thuộc vai trò thứ tư. Bài toán là ước lượng một mô hình từ dữ liệu có nhiều điểm ngoại lai; vết cận mọi tập con là bất khả thi. RANSAC lấy ngẫu nhiên một tập con nhỏ vừa đủ để xác định mô hình, đếm số điểm ủng hộ, và lặp lại. Lập luận về số lần lặp cần thiết chính là lập luận khuếch đại ở mục 3: nếu xác suất một lần lấy mẫu “sạch” là p thì sau k lần, xác suất chưa từng sạch là $(1 - p)^k$, và ta chọn k để con số đó đủ nhỏ.

7. Có cách nào biến thuật toán ngẫu nhiên thành tất định không?

| Có, và nhánh nghiên cứu đó gọi là khử ngẫu nhiên. Kỹ thuật thường dùng là thay nguồn ngẫu nhiên thật bằng một họ hàm nhỏ có tính chất tương tự — ví dụ dùng một họ hàm băm độc lập đôi một thay vì hàm ngẫu nhiên hoàn toàn, khi phân tích chỉ cần tính độc lập đôi một. Trong thực hành thì đầu, khử ngẫu nhiên hiếm khi đáng làm; giá trị của nó nằm ở lý thuyết và ở các tình huống bắt buộc phải tái lập được kết quả.

Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Trộn ngẫu nhiên một mảng đúng cách	Bài tập cài đặt	Thuật toán Fisher–Yates; kiểm bằng thống kê rằng mọi hoán vị xuất hiện đều nhau trên $n = 4$.
● Cài quicksort với chốt ngẫu nhiên và đo trên mảng đã sắp	Bài tập thực nghiệm	So với chốt cố định trên cùng dữ liệu — khác biệt sẽ rất rõ.
●● Kiểm tra nguyên tố Miller–Rabin cho số tới 10^{18}	Bài kinh điển	Chú ý phép nhân modulo tránh tràn; so thời gian với thử chia tới $\sqrt{n}$.
●● Kiểm tra $AB = C$ bằng thuật toán Freivalds	Bài kinh điển	Đo thời gian so với nhân ma trận thật; lặp 30 lần và ước lượng xác suất sai.
●● Lấy mẫu hồ chứa trên một dòng dữ liệu dài	Bài tập cài đặt	Chứng minh tính đều bằng quy nạp trước khi cài; kiểm bằng thống kê sau khi cài.
●● Tìm phần tử xuất hiện quá nửa bằng phương pháp ngẫu nhiên	Bài kinh điển	Chọn ngẫu nhiên một phần tử và kiểm; xác suất trúng $> 1/2$ nên vài chục lần là đủ. So với thuật toán bỏ phiếu tất định.
●●● Cài lát cắt nhỏ nhất của Karger	Bài kinh điển	Dùng DSU cho phép co cạnh ($B/11$); đo tỉ lệ ra đúng theo số lần lặp.
●●● Băm chuỗi với cơ số ngẫu nhiên, so với cơ số cố định trên test phá	Bài tập an ninh	Tự dựng test phá cho cơ số cố định — bài tập cho thấy rõ nhất vì sao phải ngẫu nhiên hoá.
●●● Ước lượng số nút cây quay lui bằng phương pháp Knuth	Chương C/13	Vai trò 3 của ngẫu nhiên hoá; so ước lượng với số thật trên bài n quân hậu.
●●● RANSAC ước lượng một đường thẳng từ dữ liệu có 50% điểm ngoại lai	Bài tập ứng dụng	Tính số lần lặp cần thiết trước khi chạy, rồi kiểm chứng bằng thực nghiệm.

Bài	Nguồn	Điều cần nhận ra (tiếp)
●●● Tìm kiếm cây Monte Carlo cho một trò chơi nhỏ	Bài tập cài đặt	So với duyệt đầy đủ có cắt tia (C/13); quan sát chất lượng theo số lần mô phỏng.

Kết chương và đọc tiếp

Phần C khép lại ở đây. Chương này gom lại một sợi chỉ đã chạy suốt từ phần B: ngẫu nhiên hoá không phải sự cầu may mà là cách chuyển sự phụ thuộc từ dữ liệu — thứ bạn không kiểm soát — sang các lựa chọn của chính thuật toán — thứ bạn kiểm soát. Hai điều đáng mang theo: bốn vai trò ở mục 5 để biết khi nào nghĩ tới nó, và phép tính khuếch đại cho thấy vì sao “có thể sai” lại trở thành khái niệm không có ý nghĩa thực tế sau vài chục lần lặp.

Tám chương của phần C đều trả lời câu hỏi “dùng khuôn tư duy nào để sinh ra lời giải”. Phần D đổi câu hỏi sang một hướng khác và ít dễ chịu hơn: *khi nào thì nên ngừng tìm lời giải nhanh*. Nó bắt đầu bằng hai miền có bấy số học riêng — số học và hình học — rồi tới chương quan trọng nhất của phần: cách nhận ra một bài toán là NP-khó, và bốn cách sống chung với sự thật đó (D/23).

Đọc thêm: công trình gốc (chỉ liệt kê nguồn đã qua phản biện; bản tiền ấn được ghi rõ)

Mức	Công trình và tác giả	Nơi công bố, năm	Vì sao đọc, đọc phần nào
nền tảng	Karger & Stein, “A New Approach to the Minimum Cut Problem”	Journal of the ACM, 1996	Thuật toán co cạnh ngẫu nhiên kèm mẹo đệ quy làm xác suất thành công tăng vọt. Đọc phần phân tích xác suất — ngắn, đẹp, và là mẫu mực cho việc khuếch đại.
nền tảng	Motwani & Raghavan, <i>Randomized Algorithms</i>	Cambridge University Press, 1995	Sách chuẩn của lĩnh vực; đọc chương về thuật toán Las Vegas và Monte Carlo cùng chương về bất đẳng thức tập trung.
tra khi cần	Karp & Rabin, “Efficient Randomized Pattern-Matching Algorithms”	IBM Journal of Research and Development, 1987	Băm chuỗi như một thuật toán ngẫu nhiên Monte Carlo; đọc kèm B/08.
tra khi cần	Alon, Matias, Szegedy, “The Space Complexity of Approximating the Frequency Moments”	Journal of Computer and System Sciences, 1999	Bài mở đường cho thuật toán luồng dữ liệu; đọc khi cần ước lượng thống kê với bộ nhớ nhỏ (D/24).
tra khi cần	Mitzenmacher & Upfal, <i>Probability and Computing</i>	Cambridge University Press, 2005	Dễ vào hơn Motwani–Raghavan và dạy luôn phần xác suất cần thiết.

Nguồn

- [1] R. Motwani and P. Raghavan, *Randomized Algorithms*. Cambridge University Press, 1995.